

ATIVIDADES PYTHON PANDAS

Nível 1: Identificação de Estruturas O Pandas trabalha com três estruturas principais fundamentadas em arrays Numpy. Escreva um código que importe a biblioteca Pandas e crie uma Series vazia, imprimindo seu tipo de dado padrão (dtype).

Implementação do Código:

```
import pandas as pd

# Criando uma Series vazia
series_vazia = pd.Series(dtype='float64')

# Imprimindo o tipo de dado padrão (dtype)
print(f"Tipo de dado padrão da Series vazia: {series_vazia.dtype}")
```

Nível 2: Criação de Series a partir de Listas Uma Series é um array unidimensional capaz de armazenar qualquer tipo de dado. Crie uma lista com os nomes de cinco frutas e converta essa lista em uma Series do Pandas, deixando que o Python atribua os índices numéricos automáticos (0 a 4).

Implementação do Código:

```
import pandas as pd

# 1. Criação da lista de frutas (Estrutura nativa do Python)
lista_frutas = ['Maçã', 'Banana', 'Laranja', 'Morango', 'Uva']

# 2. Conversão da lista em uma Series do Pandas
# O Pandas atribuirá automaticamente os índices de 0 a 4.
series_frutas = pd.Series(lista_frutas)

# 3. Exibição da Series resultante
print("Series de Frutas:")
print(series_frutas)
```

Nível 3: Manipulação de Índices Personalizados Ao criar uma Series, podemos definir rótulos personalizados para os eixos. Crie uma Series utilizando um array Numpy com os valores [10, 20, 30, 40], mas defina explicitamente os índices como ['A', 'B', 'C', 'D']. Imprima a Series resultante.

Implementação do Código:

```
import pandas as pd
```

```
import numpy as np
```

1. Criação do array Numpy (Eficiência de memória e processamento)

```
dados_financeiros = np.array([10, 20, 30, 40])
```

2. Definição explícita de rótulos (Labels) para os índices

```
indices_categoricos = ['A', 'B', 'C', 'D']
```

3. Construção da Series com índices personalizados

```
series_personalizada = pd.Series(dados_financeiros, index=indices_categoricos)
```

4. Exibição da estrutura resultante

```
print("Series com Índices Customizados:")
```

```
print(series_personalizada)
```

Nível 4: Uso de Dicionários e Tratamento de Dados Ausentes (NaN) Se passarmos um dicionário para o construtor `pd.Series()` e especificarmos um índice com uma chave que não existe no dicionário original, o Pandas preencherá o valor com NaN (*Not a Number*).

- Crie um dicionário: `{'jan': 100, 'fev': 200}`:

```
import pandas as pd
```

1. Definição do dicionário original (Ex: Faturamento de dois meses)

```
dados_faturamento = {'jan': 100, 'fev': 200}
```

- Crie uma Series a partir dele, mas use o índice: ['jan', 'fev', 'mar'].
2. Criação da Series com um índice expandido (Incluindo 'mar')

O Pandas tentará alinhar o dicionário ao índice fornecido.

```
series_temporal = pd.Series(dados_faturamento, index=['jan', 'fev', 'mar'])
```

- Imprima o resultado e observe o que acontece com o mês de 'mar'.

3. Exibição do resultado para análise de integridade

```
print("Série Temporal de Faturamento:")
```

```
print(series_temporal)
```

Nível 5: Estrutura Multidimensional (DataFrame) O DataFrame é uma estrutura bidimensional (tabular) de tamanho mutável. De acordo com o que foi apresentado até a página 25, ele pode ser visualizado como uma tabela SQL ou planilha. Escreva a sintaxe do construtor completo do DataFrame, listando seus 5 parâmetros principais (data, index, columns, dtype, copy) .

```
import pandas as pd
```

Sintaxe do construtor completo

```
df = pd.DataFrame(
    data=None, # Os dados brutos (Dicionário, Lista, Array, etc.)
    index=None, # Rótulos das linhas (Ex: Datas, IDs)
    columns=None, # Títulos das colunas (Ex: 'Vendas', 'Custo')
    dtype=None, # Forçar um tipo de dado específico
    copy=None # Se deve copiar os dados da origem
)
```